LLDD BE - Job 11 UpdateCompensateFromSTA

SBP Mall - ระบบประกันรายได้ | Low Level Design Document

แนวทางการใช้เอกสาร

หัวข้อ	คำอธิบาย
วัตถุประสงค์	ใช้เป็นรายละเอียดระดับพัฒนาสำหรับออกแบบ ลงมือพัฒนา ตรวจทาน และทดสอบขอบเขตที่ระบุในฉบับนี้
ลำดับการอ่าน	เริ่มจาก Overview และ Scope จากนั้นตรวจ Field/Validation, Implementation, Contract, Processing Flow, Acceptance Criteria และ Test Checklist ตามลำดับ
ผลลัพธ์ที่คาดหวัง	ผู้อ่านสามารถระบุ input, ขั้นตอนประมวลผล, output, เงื่อนไขผิดพลาด และหลักฐานการทดสอบได้จากเนื้อหาในฉบับเดียว
Java เดิม	ภาคผนวกท้ายฉบับระบุ source file, ช่วงบรรทัด และ code Java เดิมสำหรับตรวจเทียบ behavior ก่อนย้ายระบบ

คำว่า Input, Progress และ Output ในเอกสารนี้หมายถึงข้อมูลตั้งต้น ลำดับการทำงาน และผลลัพธ์ที่ตรวจสอบได้ของขอบเขตที่กำลังพัฒนา

1. บทนำ

1.1 วัตถุประสงค์

รับยอดเงินประกันรายได้จาก STA: รับยอดเงินประกันรายได้ที่ STA คำนวณกลับมา (message sta_update_compensate) แล้วเติม forecast_amount / adjust_amount และสถานะงวดลงแถวที่ Job 6 สร้างไว้ พร้อมยอดรายร้านเปิดใหม่ · แทน web service /fgiService/updateCompensateFromFS ของระบบเดิม (พอร์ตจาก ExportJdbc.updateFgiImpactStoreCompensate / insertFgiNewStoreCompensate) · ยอดนี้คือสิ่งที่ Job 8 ต้องใช้สร้างเอกสาร

1.2 ขอบเขตงาน

- Main class/script: (web service /fgiService/updateCompensateFromFS — ซอร์สไม่อยู่ใน fcsJar) / sgi-update-compensate-from-sta
- Phase: D
- Output: sgi_fgi_impact_compensations + sgi_fgi_new_store_compensations
- Estimate: 10 ชั่วโมง
- Argument รับผ่าน `INPUT` (JSON) — local ใช้ env `JOB_NAME/INPUT` · AWS Batch ใช้ `argv[3]/argv[2]` · ดูหัวข้อ 5.95 · ไม่มีตาราง job_configs และไม่มีหน้าจอควบคุม (หน้า Flow Batch Job ในกลุ่มเมนู Flow เหลือแค่ Flowchart + Database ที่ใช้ · 2026-08-06)
- ตารางเวลาตั้งที่ AWS Batch scheduled event (repo ไม่มี `@Cron`) — ยกเว้น job ที่เป็น event-driven (ดูหัวข้อ Config Schema ของ job นั้น) ซึ่งห้ามตั้ง schedule · ทุก job ถูกบันทึกลง `integration_log` โดย `main.ts` อัตโนมัติ + structured log `BATCH_START/BATCH_END` พร้อม `runId`

1.3 ข้อมูลส่งมอบ

รายการ	รายละเอียด
Track	BE
Estimate	13 ชั่วโมง = implementation 10 + unit test 3 (30%)
Owner	Aphiwit <Bank> Khammoon
Legacy class เดิม	— (ไม่มี main class เดิม)
Target repository	`SBP/srm-sps-spsap-sop-sgi-batch` (NestJS 11 + TypeORM · schema `sps_store` · มติ 2026-09-02 — ย้ายมาจาก store-backend) — batch runner ของ SBP ที่รันอยู่แล้ว 42 job บน AWS Batch · ลงทะเบียน job ใน `src/main.ts` แล้วรับ argument ผ่าน `JOB_NAME/INPUT` (local) หรือ `argv[3]/argv[2]` (AWS Batch) · ไม่ผ่าน BFF และไม่เปิด HTTP · ตารางเวลาเป็น AWS Batch scheduled event ไม่ใช่ `@Cron` · ดู `srm sps spsap sop sgi batch`

1.4 เอกสาร LLDD ที่เกี่ยวข้อง

ตารางนี้สร้างจาก endpoint และตารางที่เอกสารฉบับนี้ประกาศไว้จริง — อ่านฉบับที่อยู่ในตารางก่อนลงมือ เพื่อไม่ให้สัญญา request/response หรือชื่อคอลัมน์หลุดจากกัน

ความสัมพันธ์	เอกสาร LLDD	เกี่ยวข้องตรงไหน
สัญญากลาง	LLDD-BE-API-Common-Contracts.pdf	envelope `{success,data}` · error code · pagination · รูปแบบวันที่/เลขเอกสาร
โครงสร้างข้อมูล	LLDD-BE-Database-Structure.pdf	DDL ของตารางที่หัวข้อ Reference DB Mapping อ้างถึง
แพลตฟอร์มระบบเดิม	LLDD-BE-Integration-SBP-Platform.pdf	header จาก BFF (`x-api-key` / `x-user-*`) · การ reuse ตารางและ service ของระบบ SBP เดิม
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-API-Common-Contracts.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง
ต้องจบก่อน (ลำดับงาน)	LLDD-BE-Job-6-ExportImpactStoreToSTA.pdf	เป็นฉบับต้นทางของสัญญา/โครงที่ฉบับนี้อ้าง

2. Program Specification

2.1 การลงทะเบียน job และ Arguments (sop-sgi-batch)

งานนี้ลงทะเบียนเป็น job ชื่อ `sgi-update-compensate-from-sta` ใน `src/main.ts` ของ `SBP/srm-sps-spsap-sop-sgi-batch` โดยใช้ dispatcher เดิมของ repo (ไม่สร้าง runner ใหม่) — `main.ts` อ่านชื่อ job และ input มา 2 ทาง แล้วเลือกอัตโนมัติจากการมี `argv[3]` หรือไม่

ช่องทางรัน	ชื่อ job มาจาก	input มาจาก	ตัวอย่าง
Local / CLI / runbook	env `JOB_NAME`	env `INPUT` (JSON string)	`JOB_NAME=sgi-update-compensate-from-sta INPUT='{"dataType":"message","dataName":"sta_update_compensate","dataMessage":[{"storeCodeI":"01213","periodImpact":"6906","impactStatus":"W","forecast":"1500.00","adjust":"","compensate":[]}],"sender":"STA","sentAt":"2026-10-09T10:00:00Z"}' npm run start`
AWS Batch (ตารางเวลาจริง)	`process.argv[3]`	`process.argv[2]` (JSON string)	`node dist/main.js '{"dataType":"message","dataName":"sta_update_compensate","dataMessage":[{"storeCodeI":"01213","periodImpact":"6906","impactStatus":"W","forecast":"1500.00","adjust":"","compensate":[]}],"sender":"STA","sentAt":"2026-10-09T10:00:00Z"}' sgi-update-compensate-from-sta`

ตารางเวลาไม่ได้อยู่ในโค้ด — repo นี้ไม่มี `@Cron`/`@Interval` แม้แต่จุดเดียว (แม้ติดตั้ง `@nestjs/schedule` ไว้) cron ในหัวข้อ 5 เป็น นิยามของ AWS Batch scheduled event ที่ต้องตั้งตอน deploy ไม่ใช่ค่าที่อ่านจาก config file

Argument ที่รับได้ (`INPUT` เป็น JSON object · ไม่ส่ง = `{}`)

ระบบเดิมรับ argument แบบนี้: ไม่มี main — ระบบเดิมเป็น web service ที่ STA เรียก (`/fgiService/updateCompensateFromFS`)
ของใหม่เปลี่ยนจาก positional string เป็น JSON เพื่อให้เพิ่มฟิลด์ได้โดยไม่พังของเดิม แต่ต้องรองรับทุกอย่างที่ระบบเดิมรับได้เป็นอย่างน้อย — ห้ามลดความสามารถลง

Field	ชนิด	ไม่ส่งแล้วได้อะไร (default)	Validation	ตรงกับ argument เดิม
ซองข้อความ (`dataType` · `dataName` · `dataMessage` · `sender` · `sentAt`)	object	— (บังคับ)	`dataName = sta_update_compensate`	ของใหม่ — EAI consumer ส่งข้อความทั้งก้อนเป็น INPUT
`message` + `dryRun`	object · boolean	`dryRun = false`	ห่อซองข้อความสำหรับรันมือ	ของใหม่ — dryRun อ่านอย่างเดียว ไม่เขียนฐาน
`dryRun`	boolean	`false`	`true` = อ่าน/คำนวณครบแต่ไม่ commit และไม่ publish/ไม่ส่งไฟล์	ของกลาง ทุก job
`limit`	number	`null`	จำกัดจำนวนรายการที่ประมวลผลรอบนี้ (ใช้ตอน smoke test)	ของกลาง ทุก job

กติกาการ validate ที่ทุก job ต้องทำเหมือนกัน — parse `INPUT` ไม่สำเร็จ หรือฟิลด์ไม่ผ่าน validation ให้ log `BATCH_END` ด้วย `batchStatus: 'FAILED'` แล้ว `exit(1)` ก่อนแตะฐานข้อมูล (ห้าม fallback ไปค่า default เงียบ ๆ เมื่อผู้ใช้ตั้งใจส่งค่ามาแล้วผิด) · ฟิลด์ที่ไม่รู้จักให้ log warn แล้วข้าม ไม่ทำให้ job ล้ม

- ไม่มี cron — ทำงานเมื่อ STA publish ข้อความ · `periodImpact` เป็น `yyMM` พ.ศ. แปลงเป็น `YYYY-MM` ค.ศ. ตอนอ่าน

2.2 Input / Output

Stage	Contract for implementation
Input	ข้อความจาก STA ที่เรียก web service /fgiService/updateCompensateFromFS (storecode_i · งวด · forecast/adjust · impactStatus W/Z · ยอดรายร้านเปิดใหม่)
Progress	insert แถวงวดถ้ายังไม่มี (เฉพาะรอบ flag_action = Y) → update compensate_forecast / compensate_adjust / compensate_status ตามกติกา decode(W→I เมื่อสถานะว่างหรือ Z, Z→Z) → insert ยอดรายร้านเปิดใหม่ที่ยังไม่มี
Output	fgi_impact_store_compensate และ fgi_new_store_compensate มียอดจาก STA

2.3 ขั้นตอนการทำงาน

Step	Description
1	เริ่ม (EAI consumer สั่งรันพร้อมข้อความ)
2	ข้อความถูกต้อง? (dataName = sta_update_compensate · ฟิลด์ครบ) | No: ล้มทั้งข้อความ + อีเมล EM-07 (รายการที่ผิดรูปบางรายการถูกแยกออก ไม่ลากรายการอื่น)
3	วนทีละรายการ — 1 transaction + advisory lock ตาม ร้าน + งวด (ส่งซ้ำพร้อมกันไม่เขียนทับกันกลางทาง)
4	พบแถวงวดของรอบที่ active (flag_action Y/W)? | No: รายการนั้นไม่สำเร็จ (ไม่สร้างแถวเอง) (แถวงวดเป็นของ Job 6)
5	อัปเดต forecast_amount · adjust_amount · compensate_status (impactStatus W: สถานะว่าง/Z → I อื่นคงเดิม · Z → Z (กติกาเดิม))
6	อัปเดต/สร้างยอดรายร้านเปิดใหม่ (ไม่มีแถวแต่มีคู่ร้านในรอบ = สร้าง · ไม่มีคู่ร้าน = รายการนั้นนับเป็นไม่สำเร็จ)
7	บันทึกผลลง sgi_interface_transactions (IN · STA_UPDATE_COMPENSATE) (ส่งซ้ำ = อัปเดตแถวเดิม + retry_count · ยอดไม่ซ้ำ)
8	จบ — ครบทุกรายการ = SUCCESS · บางส่วน = PARTIAL_FAILED · ไม่สำเร็จเลย = FAILED (exit 1 + อีเมล)

2.4 เงื่อนไขตัดสิน (Decision Rules) — ตัดสินจากอะไร

Job 11 รับยอดเงินประกันรายได้ที่ STA คำนวณกลับมา (message `sta_update_compensate`) — เพิ่มกลับ 2026-10-09 แทน web service `/fgiService/updateCompensateFromFS` ของระบบเดิม (ซอร์สอยู่นอก fcsJar · กติกาพอร์ตจาก `ExportJdbc.updateFgiImpactStoreCompensate` flag FS) · ทำงานเมื่อ EAI consumer ได้ข้อความแล้วสั่ง AWS Batch ไม่มี cron

คำถามที่โค้ดต้องตอบ	ตัดสินจาก (ตาราง · คอลัมน์)	เงื่อนไขที่ต้องเป็นจริง	ไม่เข้าเงื่อนไขแล้วทำอะไร
ข้อความนี้เป็นของ job นี้หรือไม่	ซองข้อความ `dataName`	`dataName = sta_update_compensate` (ตั้งได้ที่ `SGI_JOB11_DATA_NAME`)	ไม่ตรง = ล้มทั้งข้อความ + อีเมล EM-07 · ฟิลด์ที่ไม่รู้จักถูกปฏิเสธ
รายการนี้เขียนลงงวดไหน	`storeCodeI` + `periodImpact` (yyMM พ.ศ.)	`sgi_fgi_impact_compensations` ที่ร้าน + งวดตรง และรอบ `flag_action IN ('Y','W')`	ไม่พบ = รายการนั้นไม่สำเร็จ ไม่สร้างแถวเอง (แถวงวดเป็นของ Job 6)
สถานะงวดหลังรับยอด	`impactStatus` + `compensate_status` เดิม	W: เดิมว่างหรือ Z → I · อื่นคงเดิม · Z → Z	กติกาเดิม `decode(impactStatus,'W',decode(nvl(status,'Z'),'Z','I',status),impactStatus)`
ยอดรายร้านเปิดใหม่	`compensate[]`	มีแถวแล้ว = อัปเดต · ไม่มีแถวแต่มีคู่ร้านในรอบ = สร้าง	ไม่มีคู่ร้าน = ยอดรวมยังบันทึก แต่รายการนับเป็นไม่สำเร็จ

คำถามที่โค้ดต้องตอบ	ตัดสินจาก (ตาราง · คอลัมน์)	เงื่อนไขที่ต้องเป็นจริง	ไม่เข้าเงื่อนไขแล้วทำอะไร
ผลของทั้ง job	ผลรายการทั้งหมด	ครบ = SUCCESS · บางส่วน = PARTIAL_FAILED · ไม่สำเร็จเลย = FAILED	ไม่ใช่ SUCCESS = exit 1 + อีเมลแจ้งเตือน

ค่าคงที่และโดเมนที่ใช้ในเงื่อนไขข้างบน

ทุกค่าในตารางนี้ต้องอ่านจาก config/env ไม่ hardcode ในคิวรี — แต่ ค่าตั้งต้นต้องเท่าระบบเดิมทุกตัว ไม่งั้นผลลัพธ์จะไม่ตรงกันตอนเทียบ UAT

ค่า	โดเมน / ค่าที่ระบบเดิมใช้	ที่มา
impactStatus	W = ยอดไม่เท่ากับศูนย์ · Z = ยอดเท่ากับศูนย์	สเปก STA ข้อ 3
periodImpact	yyMM พ.ศ. (เช่น 6906 = มิ.ย. 2569 = 2026-06)	สเปก STA · แปลงตอนอ่านเท่านั้น

2.5 Field, Format, and Validation

Field / UI	Format	Validation	Behavior
ตัวกระตุ้น (Trigger)	RabbitMQ exchange sgi.interface → EAI consumer → AWS Batch SubmitJob	แก้ไขได้	AWS Batch ฟังคิวเองไม่ได้ · EAI consumer bind คิว กรอง dataName = sta_update_compensate · ⧗ รอตั้งกับทีม EAI/STA
INPUT (JSON)	{ "dataType":"message", "dataName":"sta_update_compensate", "dataMessage":[…], "sender":"STA", "sentAt":"…" }	แก้ไขได้	ซองข้อความตรง ๆ · รันมือใช้ { "message": <ซอง>, "dryRun": true } · dataName ไม่ตรง = ล้มทั้งข้อความ
งวด periodImpact	yyMM พ.ศ. → YYYY-MM ค.ศ.	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	เช่น 6906 → 2026-06 · แปลงตอนอ่านข้อความ ห้ามให้ พ.ศ. เข้าตาราง sgi_*
เพดานรายการ	SGI_JOB11_MAX_ITEMS = 500	ค่าคงที่/แก้ผ่านหน้าจอไม่ได้	ข้อความควรเป็นรายร้าน — command ของ AWS Batch มีเพดานขนาด

2.6 หลักฐานที่ต้องเก็บตอนรัน

Evidence	Job-specific value	Acceptance
Input identity	ข้อความจาก STA ที่เรียก web service /fgiService/updateCompensateFromFS (storecode_i · งวด · forecast/adjust · impactStatus W/Z · ยอดรายร้านเปิดใหม่)	บันทึกไฟล์/คีย์ธุรกิจ/งวดที่รับเข้าใน run record
Output identity	fgi_impact_store_compensate และ fgi_new_store_compensate มียอดจาก STA	กระทบยอด input · สำเร็จ · ตก · ข้าม
Dedup proof	ส่งข้อความเดิมซ้ำได้ — UPDATE ทับค่าเดิม · ยอดรายร้านเปิดใหม่ INSERT ... ON CONFLICT DO NOTHING · sgi_interface_transactions upsert ตาม (data_name, direction, ร้าน, งวด) และนับ retry_count	รันซ้ำด้วย fixture เดิมต้องไม่เกิดคีย์ธุรกิจซ้ำ
Transaction proof	1 รายการ = 1 transaction + pg_advisory_xact_lock(hashtext('sgi-job11:' || ร้าน || ':' || งวด)) · รายการที่ล้มไม่ลากรายการอื่น · ไม่พบแถวงวด = ไม่สร้างแถวเอง (เป็นของ Job 6)	จำลอง error กลางทางแล้วไม่เหลือแถวค้างนอกขอบเขตที่ระบุ
Security proof	ข้อความมาจากระบบภายนอก — ตรวจ dataName · ฟิลด์ที่ไม่รู้จัก · รูปแบบรหัสร้าน/งวด/ตัวเลข ก่อนเขียน · เพดานรายการต่อข้อความ SGI_JOB11_MAX_ITEMS	config/log/error ไม่มีความลับแบบข้อความธรรมดา

2.7 โค้ด Java เดิมที่อ้างอิง

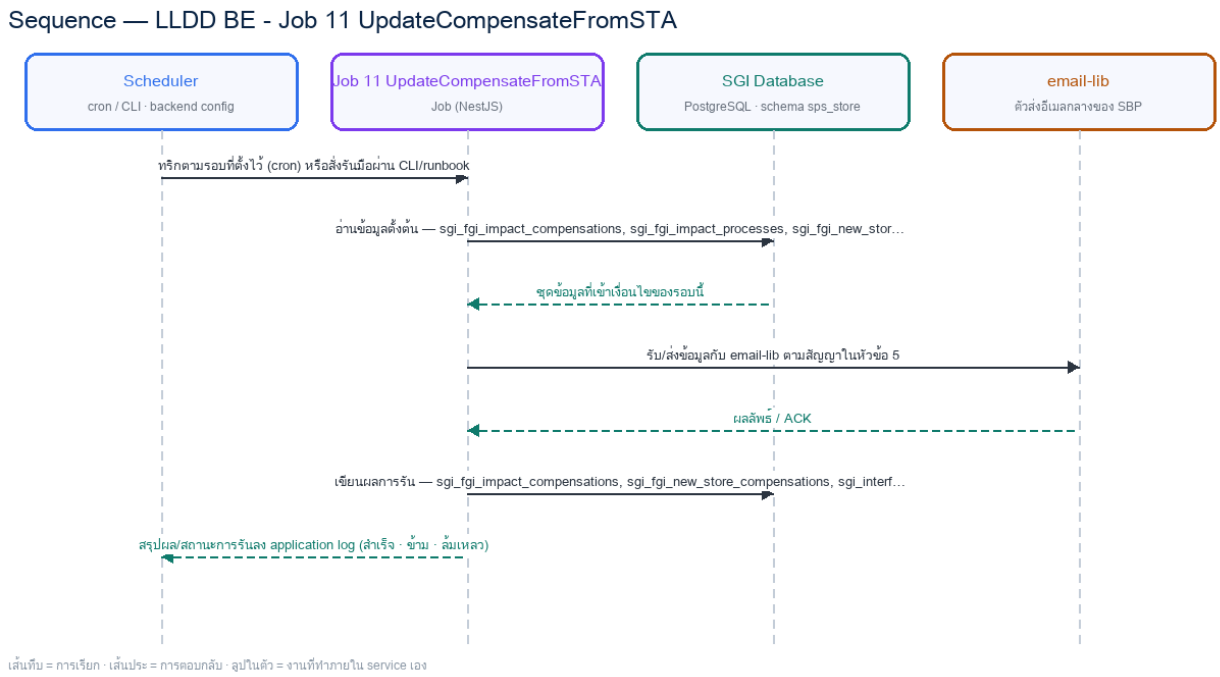
Legacy file	Line range	Responsibility to carry forward
fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java	735-860	updateFgiImpactStoreCompensate — flag FS: forecast/adjust + กติกาสถานะ W/Z (ตัวเรียก web service อยู่นอก fcsJar)
fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java	384-520	insertFgiImpactStoreCompensate — flag FS: สร้างแถวงวดเมื่อยังไม่มี
fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java	521-600	insertFgiNewStoreCompensate — flag FS: ยอดรายร้านเปิดใหม่

Line ranges refer to the legacy Java implementation under `batchjob/fcsJar/` (path นับจากราก `sbp-prototype/`). Use these ranges to preserve business behavior while implementing the target Node job.

3. Flow และ Sequence Diagram



รูปที่ 1: Implementation flow: LLDD BE - Job 11 UpdateCompensateFromSTA



รูปที่ 2: Sequence diagram: LLDD BE - Job 11 UpdateCompensateFromSTA

4. Reference DB Mapping

อ้างอิงสำหรับ implement job เท่านั้น ไม่ใช่งานออกแบบ/สร้างหน้า Database

Table / Object	R/W	Usage
sgi_fgi_impact_compensations	R/W	หาแถวงวดตามร้าน + งวด แล้วเขียนยอดและสถานะ
sgi_fgi_impact_processes	R	กรองเฉพาะรอบที่ active (flag_action Y/W)
sgi_fgi_new_store_compensations	R/W	ยอดรายร้านเปิดใหม่ (forecastN · adjustN · %)
sgi_fgi_impact_stores	R	หาคู่ร้านเมื่อยังไม่มีแถวยอดรายร้านเปิดใหม่
sgi_interface_transactions	W	ผลรายการขาเข้า (IN · STA_UPDATE_COMPENSATE)
email_template / email_sent (ระบบ SBP เดิม)	R / W (โดย email-lib)	อีเมลแจ้งเตือน EM-07 เมื่อบันทึกไม่ครบ

5. โค้ดจริงอยู่ที่ไหน

เอกสารนี้ ไม่วาง skeleton code — job ทั้ง 11 ตัว implement แล้ว การมีโค้ดสองชุด (ในเอกสาร + ของจริง) ทำให้หลุดจากกันแน่นอน · อ่านสัญญาที่หัวข้อ 2 แล้วดูของจริงตามตารางนี้

สิ่งที่ต้องดู	ที่อยู่
ชื่อ job ที่ลงทะเบียน	`sgi-job-11` ใน `SBP/srm-sps-spsap-sop-sgi-batch/src/main.ts`
service ของ job	`SBP/srm-sps-spsap-sop-sgi-batch/src/modules/sgi/job-11-*.service.ts`
เทสของ job	`SBP/srm-sps-spsap-sop-sgi-batch/src/modules/sgi/job-11-*.spec.ts`
SQL ที่ใช้จริง	อยู่ในไฟล์ service เดียวกัน (repo นี้ไม่แยก `*.sql.ts`)
คำอธิบายภาษาคนของ job	batchjob/JOB-11-* อธิบายละเอียด

6. Acceptance Criteria

- พารามิเตอร์รับผ่าน `INPUT` (JSON) และ config ของ repo — เปลี่ยนค่าโดย deploy ไม่ใช่ผ่าน API/หน้าจอ · ตารางเวลาตั้งที่ AWS Batch scheduled event ไม่ใช่ `@Cron` ในโค้ด
- การรันต้องตรวจ enabled flag ใน config · การกันรันซ้อนพึ่ง AWS Batch queue เป็นหลัก — advisory lock เป็นของใหม่ที่ต้องสร้างเองถ้างานนั้นรับความเสี่ยงรันซ้อนไม่ได้
- ทุกรอบต้องเขียน application log แบบ structured (`BATCH_START`/`BATCH_END` + `runId` — `src/main.ts` ทำให้แล้ว) และบันทึกลง `integration_log` อัตโนมัติ · error ต้องส่ง EM-07
- DB/table mapping ใช้เป็น reference สำหรับ implement Job เท่านั้น ไม่ใช่งานสร้างหน้า Database
- รองรับ rerun rule และ risk note ตาม runbook

7. Unit Test Case

11 เคส · ทุกเคสต้องรันได้โดยไม่ต่อ DB/บริการภายนอกจริง (mock ที่ขอบ repository/client) · ข้อความไทยที่ยืนยันในเทสต้องเป็น verbatim ตาม ข้อกำหนดทางธุรกิจ ห้ามพิมพ์ใหม่ · ● เทสที่ mock query runner ต้องป้อนรูป tuple `[rows, rowCount]` สำหรับ UPDATE/DELETE (กฎ #140)

อ่านข้อความ

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-01	ข้อความตามสเปก STA	`sta_update_compensate` · `periodImpact` = `6906`	อ่านได้ · งวดเป็น `2026-06` (ค.ศ.)
UT-02	● ข้อความผิด job	`dataName` = `sgi_reflow`	ล้มทั้งข้อความ + อีเมลแจ้งเตือน · exit code ไม่เป็น 0
UT-03	รายการผิดรูปปนมา	2 รายการ · รายการที่ 2 `impactStatus` = `X`	รายการที่ 1 บันทึก · รายการที่ 2 แยกออก · job `PARTIAL_FAILED`

สถานะงวด

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-04	ยอดไม่เป็นศูนย์ บนงวดใหม่	`impactStatus` = `W` · สถานะเดิมว่างหรือ `Z`	ตั้งเป็น `I`
UT-05	ยอดไม่เป็นศูนย์ บนงวดที่พิจารณาแล้ว	`impactStatus` = `W` · สถานะเดิม `A`	คงเป็น `A` (กติกาเดิม)
UT-06	ยอดเป็นศูนย์	`impactStatus` = `Z`	ตั้งเป็น `Z`

หาแถวงวด

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-07	● ไม่พบงวดของรอบที่ active	ร้าน + งวดไม่มีใน `sgi_fgi_impact_compensations` (`flag_action` Y/W)	ไม่สร้างแถวเอง · รายการนั้นไม่สำเร็จ · ไม่เขียนอะไร

ยอดรายร้านเปิดใหม่

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-08	ยังไม่มีแถวแต่มีคู่ร้านในรอบ	`storeCodeN` อยู่ใน `sgi_fgi_impact_stores` ของรอบ	สร้างแถวยอดรายร้านเปิดใหม่
UT-09	ไม่มีคู่ร้านในรอบ	`storeCodeN` ไม่อยู่ในรอบ	ยอดรวมบันทึก · รายการนับเป็นไม่สำเร็จ · interface `FAILED`

ส่งซ้ำ

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-10	ข้อความเดิมมาอีกครั้ง	ร้าน + งวดเดิม	อัปเดตทับค่าเดิม · `retry_count` +1 · ยอดไม่ซ้ำ

dryRun

Test ID	เงื่อนไข	Input	ผลลัพธ์ที่ควรได้
UT-11	รันมือแบบตรวจอย่างเดียว	{ "message": …, "dryRun": true }	อ่านแถวได้ · ไม่เขียนฐานเลย

ภาคผนวก: Java Source เดิม

Code ในส่วนนี้คัดจาก Java source เดิมโดยตรงและคงข้อความตามต้นฉบับ ใช้เลขบรรทัดที่ระบุหน้าทุก snippet เพื่อตรวจเทียบ business behavior, SQL, transaction และ error handling

J1. ExportJdbc.java — lines 735-746

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java
Original lines	735-746
Responsibility	updateFgiImpactStoreCompensate — flag FS: forecast/adjust + กติกาสถานะ W/Z (ตัวเรียก web service อยู่นอก fcsJar)

```
  public int updateFgiImpactStoreCompensate(final String NUM_WAIT_PAY, final List<ImpactStore> impactStoreList, final String flag){
int result = 0;
ImpactStore impactStore = new ImpactStore();
if(impactStoreList!=null){
if(impactStoreList.size()>0){
impactStore = (ImpactStore) impactStoreList.get(0);
}
}
String sqlAppend = "";
String sqlAppendBeforeWhere = "";
String sqlAppendWhere = "";
```

J1. ExportJdbc.java — lines 849-860

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java
Original lines	849-860
Responsibility	updateFgiImpactStoreCompensate — flag FS: forecast/adjust + กติกาสถานะ W/Z (ตัวเรียก web service อยู่นอก fcsJar)

```
sql.append(sqlAppendWhere);

/*int[] resultArray = jdbcTemplate.batchUpdate(sql.toString(), new BatchPreparedStatementSetter() {

        public void setValues(PreparedStatement ps, int i) throws SQLException {

         ImpactStore impactStore = (ImpactStore) impactStoreList.get(i);
             LogUtils.info(getClass(),"update FGI_IMPACT_STORE_COMPENSATE : IMPACT_STORE_INFO_ID="+
         impactStore.getImpactStoreInfoID()+"|STORECODE_I="+impactStore.getStoreCodeI()+
         "|MONTH="+impactStore.getCompensateMonth()+"|YEAR="+impactStore.getCompensateYear()+"");
        if(FgiConstant.FRANCHISE_STATEMENT.equalsIgnoreCase(flag)){
         ps.setString(1, impactStore.getCompensateStatus());
```

J2. ExportJdbc.java — lines 384-395

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java
Original lines	384-395
Responsibility	insertFgiImpactStoreCompensate — flag FS: สร้างแถวงวดเมื่อยังไม่มี

```
}

public int insertFgiImpactStoreCompensate(final List<ImpactStore> impactStoreList, final String flag){
int result=0;
String sqlAppend = "";
if(FgiConstant.FRANCHISE_STATEMENT.equalsIgnoreCase(flag)){
sqlAppend = " and op.storecode_i = ? "
+" and op.last_compensate_month = ? "
+" and op.last_compensate_year = ? ";
}
StringBuilder sql = new StringBuilder();
sql.append(" insert into fgi_impact_store_compensate ( ");
```

J2. ExportJdbc.java — lines 509-520

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java
Original lines	509-520
Responsibility	insertFgiImpactStoreCompensate — flag FS: สร้างแถวงวดเมื่อยังไม่มี

```
                else
                 ps.setNull(7, Types.DATE);

                if(FgiConstant.FRANCHISE_STATEMENT.equalsIgnoreCase(flag)){
                 ps.setString(8, impactStore.getStoreCodeI());
                 ps.setObject(9, impactStore.getLastCompensateMonth());
                 ps.setObject(10, impactStore.getLastCompensateYear());
                }

            }

            public int getBatchSize() {
```

J3. ExportJdbc.java — lines 521-532

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java
Original lines	521-532
Responsibility	insertFgiNewStoreCompensate — flag FS: ยอดรายร้านเปิดใหม่

```
                return impactStoreList.size();
            }
        });
result = resultArray.length;*/
ImpactStore impactStore = new ImpactStore();
if(impactStoreList!=null){
if(impactStoreList.size()>0){
impactStore = (ImpactStore) impactStoreList.get(0);
}
}
Timestamp createDate = null;
Timestamp updateDate = null;
```

J3. ExportJdbc.java — lines 589-600

รายการ	รายละเอียด
Source file	fcsJar/src/th/co/gosoft/fgi/dao/jdbc/ExportJdbc.java
Original lines	589-600
Responsibility	insertFgiNewStoreCompensate — flag FS: ยอดรายร้านเปิดใหม่

```
sql.append(" and fis.flag_verify in ('"+FgiConstant.FLAG_VERIFY_ON_PROCESS+"', '"+FgiConstant.FLAG_VERIFY_ACCEPT+"') ");
sql.append(" left join mas_store ms on ms.branch_id = fis.storecode_n ");
sql.append(" where op.flag_action = '"+FgiConstant.FLAG_VERIFY_ACCEPT+"' ");
sql.append(" and trunc(nvl(ms.close_date, to_date('01/4000','mm/yyyy'))) > to_date(op.last_compensate_year || op.last_compensate_month,'YYYYMM') ");
sql.append(sqlAppend);
sql.append(" and not exists ( ");
sql.append(" select fnsc.compensate_n_id ");
sql.append(" from fgi_new_store_compensate fnsc ");
sql.append(" where fnsc.storecode_i = op.storecode_i ");
sql.append(" and fnsc.storecode_n = fis.storecode_n ");
sql.append(" and fnsc.compensate_month = op.last_compensate_month ");
sql.append(" and fnsc.compensate_year = op.last_compensate_year ");
```